

Rúbrica de Evaluación - Semana 06

Relaciones en SQLAlchemy + Service Layer

Distribución de Puntos

Evidencia	Porcentaje	Puntos
 Conocimiento	30%	30 pts
 Desempeño	40%	40 pts
 Producto	30%	30 pts
Total	100%	100 pts

Conocimiento (30 pts)

Conceptos de Relaciones (15 pts)

Criterio	Excelente (15)	Bueno (12)	Suficiente (9)	Insuficiente (0-6)
Comprensión de relaciones	Explica 1:N y N:M con precisión, entiende ForeignKey y relationship()	Entiende ambos tipos de relación	Confunde algunos conceptos	No distingue tipos de relación

Conceptos de Service Layer (15 pts)

Criterio	Excelente (15)	Bueno (12)	Suficiente (9)	Insuficiente (0-6)
Comprensión de capas	Explica separación de responsabilidades, beneficios y cuándo aplicar	Entiende la separación Router/Service	Conoce la estructura básica	No comprende el patrón

Desempeño (40 pts)

Ejercicio 01: Relación 1:N (10 pts)

Criterio	Completo (10)	Parcial (7)	Mínimo (4)	Incompleto (0-2)
Implementación	ForeignKey, relationship(), back_populates correctos	Relación funciona con pequeños errores	Relación básica implementada	No funciona

Ejercicio 02: Relación N:M (10 pts)

Criterio	Completo (10)	Parcial (7)	Mínimo (4)	Incompleto (0-2)
Implementación	Tabla asociativa, secondary, relación bidireccional	Funciona con limitaciones	Tabla asociativa creada	No funciona

Ejercicio 03: Queries con Relaciones (10 pts)

Criterio	Completo (10)	Parcial (7)	Mínimo (4)	Incompleto (0-2)
Queries	Joins, eager loading, filtros por relación	Queries básicas funcionan	Solo queries simples	No implementa joins

Ejercicio 04: Service Layer (10 pts)

Criterio	Completo (10)	Parcial (7)	Mínimo (4)	Incompleto (0-2)
Refactorización	Lógica extraída correctamente, endpoints limpios	Services funcionales con algo de lógica en routers	Estructura creada pero incompleta	No separa responsabilidades

Producto - Blog API (30 pts)

Estructura del Proyecto (10 pts)

Criterio	Excelente (10)	Bueno (8)	Suficiente (6)	Insuficiente (0-4)
Organización	routers/, services/, models/, schemas/ correctamente separados	Estructura clara con pequeños problemas	Archivos separados pero desorganizados	Todo en un archivo

Modelos y Relaciones (10 pts)

Criterio	Excelente (10)	Bueno (8)	Suficiente (6)	Insuficiente (0-4)
Implementación	Author→Posts (1:N), Posts↔Tags (N:M) funcionando	Ambas relaciones con pequeños errores	Solo una relación funciona	No implementa relaciones

Services Implementados (10 pts)

Criterio	Excelente (10)	Bueno (8)	Suficiente (6)	Insuficiente (0-4)
Service Layer	AuthService, PostService, TagService con lógica separada	Services principales implementados	Al menos un service funcional	Sin service layer

Checklist de Entrega

Ejercicios

- ☐ Ejercicio 01 completado y funcional
- ☐ Ejercicio 02 completado y funcional
- ☐ Ejercicio 03 completado y funcional
- ☐ Ejercicio 04 completado y funcional

Proyecto Blog API

- ☐ Estructura de carpetas correcta
- ☐ Modelo Author con Posts (1:N)
- ☐ Modelo Post con Tags (N:M)
- ☐ Tabla asociativa post_tags

- ☐ AuthorService implementado
- ☐ PostService implementado
- ☐ Endpoints funcionando
- ☐ Documentación Swagger accesible

Código

- ☐ Type hints en funciones
 - ☐ Sin errores de linting
 - ☐ Código comentado donde necesario
-



Notas Adicionales

Criterios de Aprobación

- Mínimo **70%** del puntaje total (70 pts)
- Al menos **50%** en cada sección
- Proyecto funcional y ejecutable

Penalizaciones

- Código sin type hints: -5 pts
- Errores de linting no corregidos: -5 pts
- Entrega tardía: -10 pts por día

Bonificación

- Tests unitarios para services: +5 pts
- Documentación extra en código: +3 pts